ChatGPT

TV-Optimized Countdown Display for Church Live Stream

Create a **full-screen countdown page** with large, high-contrast text and clear messaging. Televisions often crop edges (overscan), so keep all content within a 5% "safe area" margin [1]. Use a simple layout: e.g. a big, bold countdown clock (hours:minutes:seconds) centered on screen and a sidebar or overlay of text messages. Follow TV UI design principles – ensure text is legible from ~10 feet away with very large fonts, readable colors and minimal clutter [2] [1]. For example, set a dark background with bright white or gold text (avoiding pure saturated reds/yellows) to be gentle on the eyes [3] [4]. The Marvel guideline suggests at least 60px top/bottom and 90px side margins on a 1920×1080 canvas [1]. Don't overload the screen – keep info density low, use simple icons or shapes, and group related instructions together. Use clear labels like "Prayer begins in…" or "Live starts in…" instead of ambiguous time strings [5] [2]. Always test the design on an actual TV or monitor to verify visibility and overscan safety [6].

Design for TV: a safe-area margin diagram (source: Marvel app) shows keeping text well inside the edges [1] [7]. A solid dark background with bold white text and simple graphics maximizes contrast. Avoid busy backgrounds and flashing elements to prevent distraction or visual strain [4] [3]. Use CSS to center content (e.g. Flexbox) and scale fonts with viewport units (e.g. `font-size: 8vw;` *) for consistent sizing across displays.*

Immersive Countdown and Notice Board Design

Make the countdown *immersive* and thematic. For example, you could overlay a semi-transparent church-related image or subtle motion (slow zoom on a cross or warm stained-glass background) behind the timer to create atmosphere. Keep the countdown numerals highly prominent (using a decorative yet clear font) and consider a subtle animation (like a gentle fade or "flip" effect on digits) to draw attention.

StreamingChurch.tv's example shows "EPIC COUNTDOWN BEGINS!" in bold graphics – you can mimic that style with your own branding or motivational tagline [8].

For the "notice board" messages, present a clean list or carousel of bullet-point instructions. Frame this list like a bulletin board by using bordered boxes or pinned-note graphics in CSS. Use concise language with friendly icons (e.g. 🕊 "Please pray quietly," 🔇"Silence phones," 📖"Have your Bible ready," 🗣"No side conversations," 📷"Check your camera & mic"). You can either display multiple points at once as a static side panel or cycle through them one at a time. For instance, use a JavaScript timer to rotate message text every 10–20 seconds, fading out the old message and fading in the next, so viewers notice each tip in turn. This mimics a digital bulletin board updating over time. Make the notice text large and high-contrast, with icons or borders to group "Dos" and "Don'ts."

Example countdown style: bold, central timer with a clear message ("EPIC COUNTDOWN BEGINS!") and dynamic background (source: StreamingChurch.tv) [8]. A similar approach can be used for Sunday school: a striking title with the countdown, plus rotating instructions on the side or bottom. Use smooth CSS transitions for the messages (e.g. `opacity` *fades or slide-ins) to keep the display visually engaging but not jarring.*

Implementation Steps (HTML, CSS, JavaScript)

1. **HTML Structure:** Create a simple HTML page with elements for the countdown and notices. For example:

   ```
   <div id="container">
     <div id="countdown">08:30:00</div>
     <div id="notice">Welcome – get ready for worship!</div>
   </div>
   ```

   Use semantic tags or ARIA roles (e.g. `role="timer"` on the countdown) so screen readers can identify it [9].

2. **CSS Styling:** In your CSS, set `html, body { height: 100%; margin: 0; padding: 0; }` and use a dark background (`body { background: #001; }`). Center content with Flexbox: `display: flex; align-items: center; justify-content: center; flex-direction: column;`. Assign the countdown a huge font (e.g. `font-size: 8vw; font-weight: bold; color: white;`) and the notice a slightly smaller font (e.g. `font-size: 4vw; color: lightgoldenrodyellow; margin-top: 2vh;`). Apply the "safe area" margins via padding on the container (e.g. `padding: 60px 90px; box-sizing: border-box;`). For message transitions, define CSS keyframes or simple classes (e.g. a `.fade-in { animation: fadeIn 1s ease-in; }`).

3. **JavaScript Countdown:** Use `Date` objects and `setInterval` to update the timer every second. On page load, compute the target time (Sunday 9:50 AM) for this week. For example:

```
let now = new Date();
// Calculate upcoming Sunday at 9:50 AM
let target = new Date(now);
let day = now.getDay();  // Sunday=0, Saturday=6
let daysUntilSunday = (7 - day) % 7;
// If today is Sunday and before 8:30, use today; if after 9:50, schedule
next week
if (day === 0 && now.getHours() < 8) {
  daysUntilSunday = 0;
} else if (day === 0 && now.getHours() >= 9 && now.getMinutes() > 50) {
  daysUntilSunday = 7;
}
target.setDate(now.getDate() + daysUntilSunday);
target.setHours(9, 50, 0, 0);  // 9:50:00 AM
```

Then start an interval:

```
let countdownEl = document.getElementById('countdown');
setInterval(() => {
  now = new Date();
  let diff = target - now;
  if (diff <= 0) {
    countdownEl.textContent = "00:00:00";
    clearInterval(...);
    // Optionally display "LIVE NOW!" or similar
    return;
  }
  let hrs = Math.floor(diff/3600000);
  let mins = Math.floor((diff % 3600000)/60000);
  let secs = Math.floor((diff % 60000)/1000);
  countdownEl.textContent =
    String(hrs).padStart(2,'0') + ':' +
    String(mins).padStart(2,'0') + ':' +
```

```
    String(secs).padStart(2,'0');
}, 1000);
```

This updates the `<div>` text every second. If you want separate hours/minutes/seconds elements, adjust accordingly.

4. **Scheduling Messages:** Define an array of notice texts and cycle them. For example:

```
const notices = [
  "□ Please pray quietly and prepare in silence.",
  "□Have your Bible and notebook ready.",
  "□Phones off: Silence all devices now.",
  "□No side conversations or games, please.",
  "□Check your internet and audio settings."
];
let idx = 0;
let noticeEl = document.getElementById('notice');
setInterval(() => {
  noticeEl.classList.add('fade-out');
  setTimeout(() => {
    idx = (idx+1) % notices.length;
    noticeEl.textContent = notices[idx];
    noticeEl.classList.remove('fade-out');
    noticeEl.classList.add('fade-in');
  }, 1000);
}, 15000);
```

Here, messages change every 15 seconds with a fade-out/fade-in animation. Adjust the timing as needed. Ensure text stays visible long enough to read. Use concise bullet-style phrasing (the above are examples). Include both "do" and "don't" items clearly, and consider repeating an important one (like silencing phones) to reinforce it.

5. **Assets and Media:** For extra polish, include a subtle background audio track (instrumental or hymns) that plays during the countdown and fades out at 9:50, as StreamingChurch.tv suggests pairing music or announcements with the countdown [8]. This can be done with an `<audio>` element set to autoplay and loop, or controlled via JavaScript. (Be mindful of volume and test that it isn't jarring to worshippers.)

Deployment and TV Setup

Host the page as a **static site**. You can simply run it from a USB drive or local machine if internet is unavailable, or deploy via GitHub Pages, Netlify, etc. to be accessible via browser. Connect a computer or streaming device (like an Amazon Fire Stick or ChromeCast in full-screen mode) to the TV's HDMI. Launch a modern browser (Chrome/Firefox/Edge) in full-screen kiosk mode to display the page. Disable any screensaver or sleep settings. Using a dedicated streaming PC (per StreamingChurch.tv advice) can improve

reliability [8]. If possible, test on the actual TV hardware beforehand: different TVs may render colors and fonts slightly differently, and overscan can still crop part of edges [6].

Accessibility and Testing

While the audience is a church congregation, applying basic accessibility ensures no one is inadvertently excluded. Use **high contrast** (e.g. white on black) and large sans-serif fonts to aid legibility [4] [2]. Avoid blinking or very rapid animations to prevent discomfort [4]. Provide **ARIA labels** (e.g. `aria-live="polite"` on the countdown) so that screen readers can announce "X minutes remaining" if someone is visually impaired [9]. You can even include an off-screen element that updates with plain text like "Next live stream starts in … minutes" for screen readers. Include alt text or captions if you use any icons or images. Test the page with browser zoom to ensure text remains readable, and try a screen reader or high-contrast mode.

Automate basic tests by simulating different times: temporarily set your computer's clock to just before 8:30 AM on a Sunday to see the countdown start, and to 9:50 to ensure it stops or switches correctly. Also test edge cases: what if someone opens the page on Monday? Ideally it should show next Sunday's countdown or a "Next live in X days" message. Finally, run the site on the TV (or a large monitor) under the actual lighting conditions of the room to confirm brightness and contrast. Meeting these design and technical guidelines will ensure the countdown display is beautiful, user-friendly, and reliable for your Sunday school audience [1] [4] [9].

Sources: TV UI design guidelines recommend large fonts, high contrast and safe margins [2] [1]. Church livestream experts emphasize using a professional pre-stream countdown with music/announcements and clear notices [8]. Accessibility best-practices include semantic labels and non-flashing, high-contrast designs [4] [9]. These principles guide our detailed countdown implementation.

---

[1] Designing for Television - TV UI's - Marvel Blog
https://marvelapp.com/blog/designing-for-television/

[2] [3] [6] [7] Beginner's Guide to Smart TV UI Design and Prototyping
https://learn.protopie.io/blog/smart-tv-design-and-prototyping

[4] [5] How Live Events Producers Use Countdown Timers for Agendas
https://www.letstimeit.com/countdown-timers-and-accessibility-designing-for-inclusivity-in-live-events/

[8] 10 Easy Improvements for Your Church Live Stream – StreamingChurch.tv Blog
https://blog.streamingchurch.tv/top-10-easy-improvements-to-your-church-live-stream/

[9] 15 Pure CSS Countdown Timer Examples
https://freefrontend.com/css-countdowns/